

AlgoQuestVR: Interactive VR Puzzles for Learning Sorting Algorithms

Archit Gupta*

North Carolina State University

Aryan Sharma Inguva†

North Carolina State University

Ryan Mikula‡

North Carolina State University

ABSTRACT

Conventional algorithm visualizations are largely passive 2D animations; learners can recount steps yet struggle to apply them to novel inputs. Although virtual reality (VR) affords spatial presence and embodied interaction, there is limited work that couples algorithm instruction with interactive practice and code-level grounding. AlgoQuestVR reframes core sorting algorithms as short, replayable VR puzzle levels. A hub world lets learners select Bubble Sort or Quick Sort, and within each algorithm they can enter either a guided “demo” or a more open-ended “puzzle” mode. The learner manipulates a rack of spheres using ray or direct grab, with snapping feedback when values are placed in correct order. A world-space pseudocode panel highlights the currently executed line in green and explains, in plain English, which elements are being compared and whether they will be swapped. We report formative, in-class user testing with four participants, summarize qualitative feedback, and describe iterations made to improve usability and clarity.

1 INTRODUCTION

Algorithm visualizations have a long history in computing education, yet most systems rely on passive 2D animations that help learners recount steps but fail to support transfer to novel inputs, code-level reasoning, or debugging. Virtual reality offers spatial presence and embodied manipulation, creating opportunities to transform algorithms into interactive puzzles rather than linear videos. However, existing VR learning experiences rarely connect hands-on interaction to the underlying source code that students ultimately need to understand. AlgoQuestVR addresses this gap for sorting algorithms by combining guided walkthroughs, where learners can scrub through Bubble Sort and Quick Sort step-by-step, with replayable puzzle levels that require learners to complete the algorithms with gradually reduced scaffolding. At the core of the system, every physical action in the virtual environment, such as comparing or swapping spheres, is explicitly linked to a highlighted line of pseudocode in world space, reinforcing how algorithmic operations map to code structure.

2 SYSTEM OVERVIEW

AlgoQuestVR is implemented in Unity 6 using the XR Interaction Toolkit, combining ray-based and direct-grab interactions with XR socket interactors for placement. The application is organized into a modular scene structure: a central hub and distinct Demo and Puzzle scenes for Bubble Sort and Quick Sort. Each scene instantiates its UI panels, sorting rack, and sphere elements at runtime through C# scripts.

At the core of the system is a step-wise algorithm engine that emits structured events for comparisons and swaps. In Demo mode,

these events trigger animations, pseudocode highlighting, and explanatory text updates. Puzzle mode uses the same engine but evaluates learner actions against the next valid algorithmic step rather than presenting scripted playback.

Spheres representing array values are generated by shuffling a 1- N sequence and assigning floating text to each element. Visual cues communicate algorithm state: comparison highlights, incorrect-step flashes, and a final all-green state for completion. Real-time correctness checks occur through socket interactions, allowing the system to revert invalid puzzle actions to the last valid configuration.

Scene transitions are handled through a secondary navigation panel that loads the appropriate mode or returns the learner to the hub. Overall, the architecture enables a tightly coupled relationship between algorithmic logic, visual feedback, and interaction, while keeping each mode decoupled through separate scenes and runtime-generated content.

3 USER EXPERIENCE WALKTHROUGH

Learners begin in a calm hub environment featuring two clear entry points: Bubble Sort and Quick Sort. Approaching either option reveals two modes. *Demo* offers a guided introduction, while *Puzzle* provides a more autonomous challenge. Selecting a mode transports the learner into a focused workspace designed for that algorithm.

In Demo mode, learners stand before a rack of spheres arranged in a randomly ordered sequence. To the left is a simple control panel with buttons that allow them to play through the algorithm, advance one step at a time, reshuffle the data, or reset the scene. As the algorithm progresses, spheres highlight when compared and move when swapped, while the world-space pseudocode panel indicates the active line alongside a short plain-language description. Although learners may pick up and inspect spheres at any time, the demo provides a primarily guided, observational experience.

Puzzle mode shifts agency to the learner. The rack loads another randomized sequence, but no stepwise animation accompanies it. Instead, the learner performs each comparison or swap themselves by moving spheres into position. Immediate visual feedback signals whether the action matches the expected algorithmic step, encouraging reasoning about order and algorithm flow. As the learner progresses, the final sorted state is marked by a unified green highlight.

Across both modes, navigation remains simple: a panel positioned behind the learner enables movement back to the hub or into another algorithm, maintaining a smooth loop between exploration, practice, and re-entry.

4 INTERACTION AND FEEDBACK DESIGN

AlgoQuestVR uses a small set of consistent interactions and visual cues to keep attention on algorithmic reasoning rather than interface learning. To support different comfort levels in VR, users can manipulate elements either through ray-based grabbing or direct hand grabbing. All element placements are constrained to discrete rack positions, which helps learners reason in terms of array indices rather than free-form spatial placement.

The system relies on lightweight, state-driven visual feedback to communicate what the algorithm is doing at each moment. Elements being compared are highlighted in yellow to draw attention

*e-mail: agupta85@ncsu.edu

†e-mail: ainguva@ncsu.edu

‡e-mail: rmmikula@ncsu.edu



Figure 1: Quick Sort Demo Scene

to the active pair, reinforcing the idea that sorting proceeds through local comparisons. During Puzzle mode, incorrect actions trigger a brief red flash to clearly signal failure without requiring the learner to interpret text-heavy error messages. When the rack reaches a fully sorted configuration, all elements turn green to provide a clear completion state.

To encourage iteration rather than punishment, the puzzle design emphasizes fast recovery: invalid steps revert the rack to the last valid configuration so learners can immediately retry and refine their mental model. In Demo mode, interaction remains exploratory—learners can still pick up elements freely—while the control panel enables rapid resets and reshuffles so the same concept can be practiced across multiple permutations. Together, these choices aim to make algorithm structure visible through interaction, and mistakes informative rather than discouraging.

5 EXPERIMENTS

We conducted formative, in-class testing with four participants who were introduced to AlgoQuestVR and asked to try both Demo and Puzzle modes for Bubble Sort and Quick Sort. The goal of this pilot was to assess basic usability, clarity of the hub/menu flow, and whether the interaction model felt intuitive for first-time users.

Overall feedback was positive: participants reported that grabbing and repositioning elements made the sorting process easier to follow than a typical 2D animation, and that the demo controls supported step-by-step understanding. Participants also provided concrete suggestions for improving usability. Two consistent themes were (1) improving scene lighting to make the rack and UI panels easier to read at a glance, and (2) reordering items in the hub/menu to make the Demo/Puzzle choices more immediately discoverable.

Following this feedback, we iterated on the application by adjusting lighting to improve readability and contrast, and restructuring the hub menu flow to present algorithm and mode choices more clearly. These changes were incorporated into the current version of AlgoQuestVR.

6 CONCLUSION

AlgoQuestVR demonstrates how VR can turn sorting algorithms into interactive, replayable learning tasks while preserving a connection to algorithm structure through step-linked cues. Our formative in-class testing with four users suggests that embodied manipulation (grabbing, comparing, and placing elements) can make algorithm behavior feel more intuitive, and that simple step controls support learners in building confidence before attempting puzzle-style practice. The same pilot also surfaced practical usability issues—particularly lighting and menu ordering—which we addressed through iterative refinements. Together, these results motivate continued development of VR-first algorithm learning experiences that combine clarity, interactivity, and lightweight scaffolding.

7 FUTURE WORK

Future work will expand AlgoQuestVR beyond Bubble Sort and Quick Sort to additional algorithm families (e.g., insertion/merge sorting and introductory graph traversals), and explore richer scaffolding that adapts to learner errors (e.g., context-sensitive hints or step explanations that appear only when needed). We also plan a larger, structured user study with learning-focused measures to evaluate whether the experience improves transfer to code tracing and problem solving compared to standard visualizations.

ACKNOWLEDGMENTS

We thank the students who participated in our formative testing sessions and provided thoughtful feedback that shaped the interaction and menu design of AlgoQuestVR. We are also grateful to the teaching staff and course coordinators who supported the deployment of early prototypes in the classroom. Special thanks to colleagues who offered guidance on VR interaction patterns and usability considerations, as well as to peers who assisted with scene construction, lighting adjustments, and debugging during iterative development. This project benefited from the encouragement and informal review of many collaborators, whose insights helped refine both the educational framing and technical implementation.

REFERENCES

- [1] N. F. Imam, K. Shaker, and C. Batty. Creating tangible vr spaces for exploring algorithm complexity and data structures. In *2023 IEEE International Symposium on Mixed and Augmented Reality Adjunct (ISMAR-Adjunct)*, pp. 118–123, 2023. doi: 10.1109/ISMAR-Adjunct60411.2023.00035 2
- [2] M. G. Khan and H. N. Qureshi. Next-gen classrooms: Augmented and virtual reality in modern education. *IEEE Access*, 12:66747–66759, 2024. doi: 10.1109/ACCESS.2024.10560399 2

[2] [1]